Здесь будет титульник, листай ниже

СОДЕРЖАНИЕ

1 ПОСТАНОВКА ЗАДАЧИ.....5
1.1 Описание входных данных.....7
1.2 Описание выходных данных.....7
2 МЕТОД РЕШЕНИЯ.....8
3 ОПИСАНИЕ АЛГОРИТМОВ.....10
3.1 Алгоритм конструктора класса Items.....10
3.2 Алгоритм конструктора класса Items.....10
3.3 Алгоритм конструктора класса Items.....11
3.4 Алгоритм деструктора класса Items.....11
3.5 Алгоритм метода increaseDurability класса Items.....11
3.6 Алгоритм метода decreaseDurability класса Items.....12
3.7 Алгоритм метода getName класса Items.....12
3.8 Алгоритм метода getDurability класса Items.....13
3.9 Алгоритм функции repairItem.....13
3.10 Алгоритм функции damageItem.....13
3.11 Алгоритм функции main.....14
3.12 Алгоритм функции createBrokenItem.....15
4 БЛОК-СХЕМЫ АЛГОРИТМОВ.....16
5 КОД ПРОГРАММЫ.....22
5.1 Файл Items.cpp.....22
5.2 Файл Items.h.....23
5.3 Файл main.cpp.....24
6 ТЕСТИРОВАНИЕ.....25
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....26

1 ПОСТАНОВКА ЗАДАЧИ

В компьютерной игре существует система предметов. Каждый предмет имеет имя и показатель прочности. В процессе работы программы предмет может быть отремонтирован, повреждён или создан как сломанный объект.

Одни операции должны изменять исходный предмет, другие — работать только с его копией. Также необходимо предусмотреть ситуацию, при которой сломанный предмет может отсутствовать.

Требуется разработать программную систему, содержащую один класс. Класс должен включать:

- одно закрытое свойство строкового типа — имя предмета;
- одно закрытое свойство целочисленного типа — прочность предмета;
- конструктор по умолчанию (устанавливает по-умолчанию имя предмета Unknown и прочность 0);
- конструктор с параметрами для установки имени и начальной прочности объекта;
- конструктор копии;
- деструктор;
- метод увеличения прочности на заданное значение;
- метод уменьшения прочности на заданное значение;
- метод получения имени предмета;
- метод получения прочности предмета.

Конструкторы и деструктор должны выводить сообщения в следующем формате:

- Item (Cnstr): <имя предмета>
- Item (ParamCnstr): <имя предмета>

- Item (CopyCnstr): <имя предмета>
- ~Item (Dcnstr): <имя предмета>

Требуется реализовать следующие функции:

- Функция, принимающая объект **по ссылке** и увеличивающая его прочность на 10.
- Функция, принимающая объект **по значению** и уменьшающая его прочность на 5. Внутри функции повреждения вывести строку: "Inside damageItem: <имя> durability = <прочность>";
- Функция создания сломанного предмета принимающая в качестве параметра целое значение - режим работы, возвращающая **указатель на объект**:
    - если введён режим 0, функция должна вернуть нулевой указатель;
    - в противном случае функция должна создать динамический объект со значениями:
        - имя: Broken
        - прочность: 0и вернуть указатель на него.

Алгоритм работы программы

1. Ввести имя предмета, начальную прочность и режим работы.
2. Создать объект с использованием конструктора с параметрами.
3. Передать объект в функцию ремонта.
4. Вывести:After repair: <имя> durability = <прочность>
5. Передать объект в функцию повреждения.
6. После возврата из функции вывести:After damage: <имя> durability = <прочность>
7. Вызвать функцию создания сломанного предмета.
    7.1. Если возвращен нулевой указатель, вывести:Broken pointer

is nullptr

7.2. Иначе вывести:Broken item: <имя сломанного предмета> durability = <прочность сломанного предмета>

8. Освободить динамическую память.

9. Завершить работу программы.

1.1 Описание входных данных

Пример ввода:

```
Sword 50 1
```

1.2 Описание выходных данных

Пример вывода:

```
Item (ParamCnstr): Sword
After repair: Sword durability = 60
Item (CopyCnstr): Sword
Inside damageItem: Sword durability = 55
~Item (Dcnstr): Sword
After damage: Sword durability = 60
Item (ParamCnstr): Broken
Broken item: Broken durability = 0
~Item (Dcnstr): Broken
~Item (Dcnstr): Sword
```

2 МЕТОД РЕШЕНИЯ

Для решения задачи используется:

- объект obj класса Items предназначен для демонстрации передачи объекта по ссылке и по значению;
- объект itemPtr класса Items предназначен для демонстрации работы с динамической памятью через указатель;
- функция repairItem для реализации увеличения прочности;
- функция damageItem для реалиазации уменьшения прочности;
- функция createBrokenItem для динамического создания объекта в памяти через указатель.

Класс Items:

- свойства/поля:
  - поле название предмета:
    - наименование — name;
    - тип — string;
    - модификатор доступа — private;
  - поле прочность предмета:
    - наименование — durability;
    - тип — int;
    - модификатор доступа — private;
- функционал:
  - метод Items — конструктор по умолчанию;
  - метод Items — конструктор с параметрами;
  - метод Items — конструктор копии;
  - метод ~Items — деструктор;
  - метод increaseDurability — увеличение прочности на заданное

значение;

- метод decreaseDurability — уменьшение прочности на заданное значение;
- метод getName — получение имени предмета;
- метод getDurability — получение прочности предмета.

3 ОПИСАНИЕ АЛГОРИТМОВ

Согласно этапам разработки, после определения необходимого инструментария в разделе «Метод», составляются подробные описания алгоритмов для методов классов и функций.

3.1 Алгоритм конструктора класса Items

Функционал: устанавливает по умолчанию имя предмета Unknown и прочность 0.

Параметры: нет.

Алгоритм конструктора представлен в таблице 1.

Таблица 1 – Алгоритм конструктора класса Items

№	Предикат	Действия	№ перехода
1		Присвоить name значение "Unknown"	2
2		Присовить durability значение 0	3
3		Вывод "Item (Cnstr): <имя предмета>"	∅

3.2 Алгоритм конструктора класса Items

Функционал: Для создания и установки имени и начальной прочности объекта.

Параметры: строковый n, целочисленный d.

Алгоритм конструктора представлен в таблице 2.

Таблица 2 – Алгоритм конструктора класса Items

№	Предикат	Действия	№ перехода
1		Присвоить name значение параметра n	2

№	Предикат	Действия	№ перехода
2		присвоить durability значение пареметра d	3
3		Вывод "Item (ParamCnstr): <имя предмета>"	∅

3.3 Алгоритм конструктора класса Items

Функционал: для создания и установки имени и начальной прочности объекта.

Параметры: ссылка на объект other класса Items.

Алгоритм конструктора представлен в таблице 3.

Таблица 3 – Алгоритм конструктора класса Items

№	Предикат	Действия	№ перехода
1		Присвоить name значение параметра name объекта other	2
2		Присвоить durability значение параметра durability объекта other	3
3		Вывод "Item (CopyCnstr): <имя предмета>"	∅

3.4 Алгоритм деструктора класса Items

Функционал: деструктор.

Параметры: нет.

Алгоритм деструктора представлен в таблице 4.

Таблица 4 – Алгоритм деструктора класса Items

№	Предикат	Действия	№ перехода
1		Вывод "~Item (Dcnstr): <имя предмета>"	∅

3.5 Алгоритм метода increaseDurability класса Items

Функционал: увеличение прочности предмета на параметр value.

Параметры: целочиселнный value.

Возвращаемое значение: нет.

Алгоритм метода представлен в таблице 5.

Таблица 5 – Алгоритм метода increaseDurability класса Items

№	Предикат	Действия	№ перехода
1		Присвоить durability значение durability + value	∅

3.6 Алгоритм метода decreaseDurability класса Items

Функционал: уменьшение прочности предмета на параметр value.

Параметры: целочисленный value.

Возвращаемое значение: нет.

Алгоритм метода представлен в таблице 6.

Таблица 6 – Алгоритм метода decreaseDurability класса Items

№	Предикат	Действия	№ перехода
1		Присвоить durability значение durability - value	∅

3.7 Алгоритм метода getName класса Items

Функционал: получение имени предмета.

Параметры: нет.

Возвращаемое значение: строковый name.

Алгоритм метода представлен в таблице 7.

Таблица 7 – Алгоритм метода getName класса Items

№	Предикат	Действия	№ перехода
1		возврат name	∅

3.8 Алгоритм метода getDurability класса Items

Функционал: возвращает прочность предмета.

Параметры: нет.

Возвращаемое значение: целочисленный durability.

Алгоритм метода представлен в таблице 8.

Таблица 8 – Алгоритм метода getDurability класса Items

№	Предикат	Действия	№ перехода
1		возврат durability	∅

3.9 Алгоритм функции repairItem

Функционал: увеличение прочности на 10 ед..

Параметры: ссылка на объект other класса Items.

Возвращаемое значение: нет.

Алгоритм функции представлен в таблице 9.

Таблица 9 – Алгоритм функции repairItem

№	Предикат	Действия	№ перехода
1		Вызов метода obj.increaseDurability(10)	∅

3.10 Алгоритм функции damageItem

Функционал: уменьшение прочности на 5 ед. и вывод информации о предмете.

Параметры: объект other класса Items.

Возвращаемое значение: нет.

Алгоритм функции представлен в таблице 10.

Таблица 10 – Алгоритм функции damageItem

№	Предикат	Действия	№ перехода
1		Вызов метода obj.decreaseDurability(5)	2
2		Вывод "Inside damageItem: <имя> durability = <прочность>"	∅

3.11 Алгоритм функции main

Функционал: основная функция программы.

Параметры: нет.

Возвращаемое значение: целочисленный, код возврата.

Алгоритм функции представлен в таблице 11.

Таблица 11 – Алгоритм функции main

№	Предикат	Действия	№ перехода
1		инициализация строки n	2
2		инициализация целочисленных переменых d и w	3
3		Ввод n, d, w	4
4		инициализация объекта obj с параметрами n и d	5
5		Вызов функции repairItem с параметром ссылки на объект obj	6
6		Вывод "After repair: <имя> durability = <прочность>"	7
7		Присвоить указателю на объект itemPtr класса Items значение метода createBrokenItem с параметром w	8
8	itemPtr == nullptr	Вывод "Broken pointer is nullptr"	11
			9
9		Вывод "Broken item: <имя сломанного предмета>	10

№	Предикат	Действия	№ перехода
		durability = <прочность сломанного предмета>"	
10		Удаление указателя на объект itemPtr класса Items	11
11		Возврат 0	∅

3.12 Алгоритм функции createBrokenItem

Функционал: Создание нового объекта в динамической памяти.

Параметры: целочисленный work.

Возвращаемое значение: указатель на созданный объект obj или nullptr.

Алгоритм функции представлен в таблице 12.

Таблица 12 – Алгоритм функции createBrokenItem

№	Предикат	Действия	№ перехода
1	work == 0	вернуть nullptr	∅
			2
2		создать указатель obj класса Items и присвоить адрес нового объекта, выделенного в динамической памяти с парметрами "Broken" и 0	3
3		вернуть obj	∅

4 БЛОК-СХЕМЫ АЛГОРИТМОВ

Представим описание алгоритмов в графическом виде на рисунках 1-6.

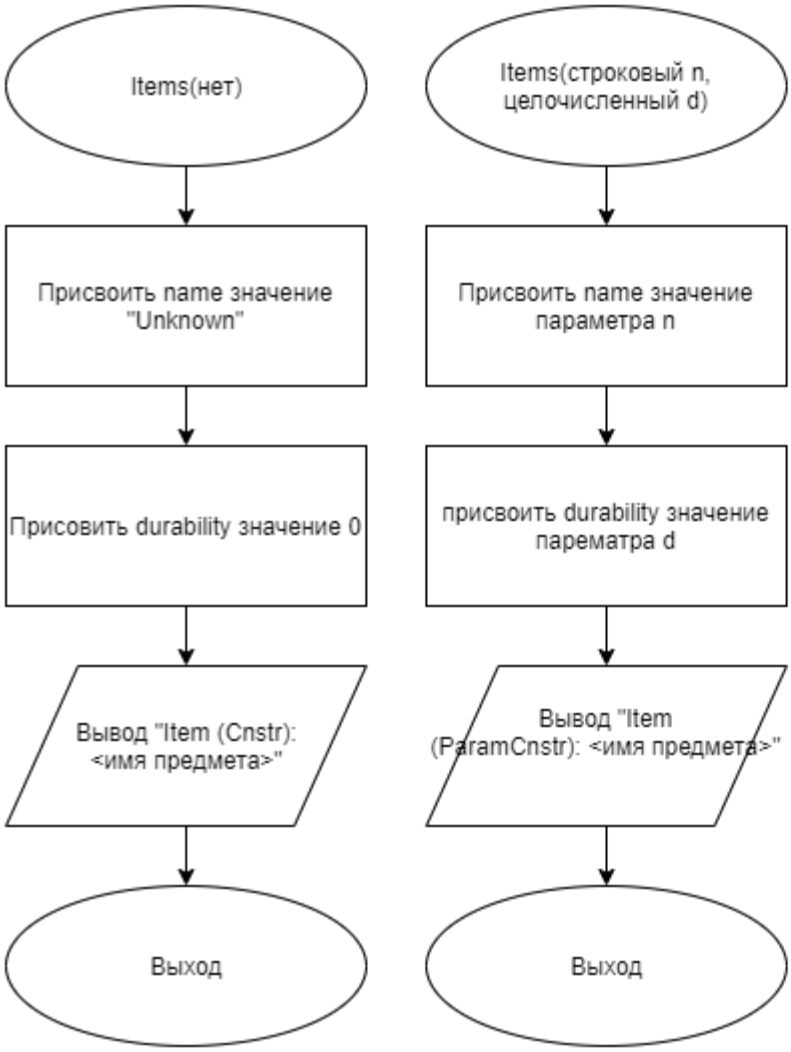

Рисунок 1 – Блок-схема алгоритма

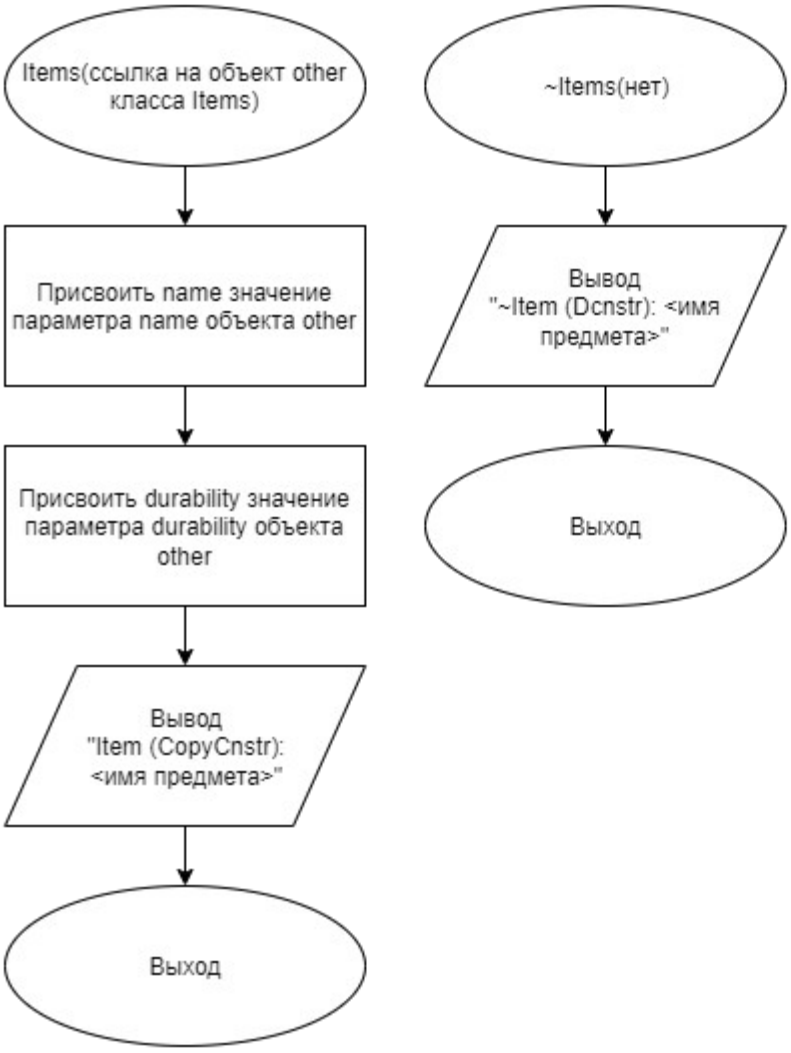

Рисунок 2 – Блок-схема алгоритма

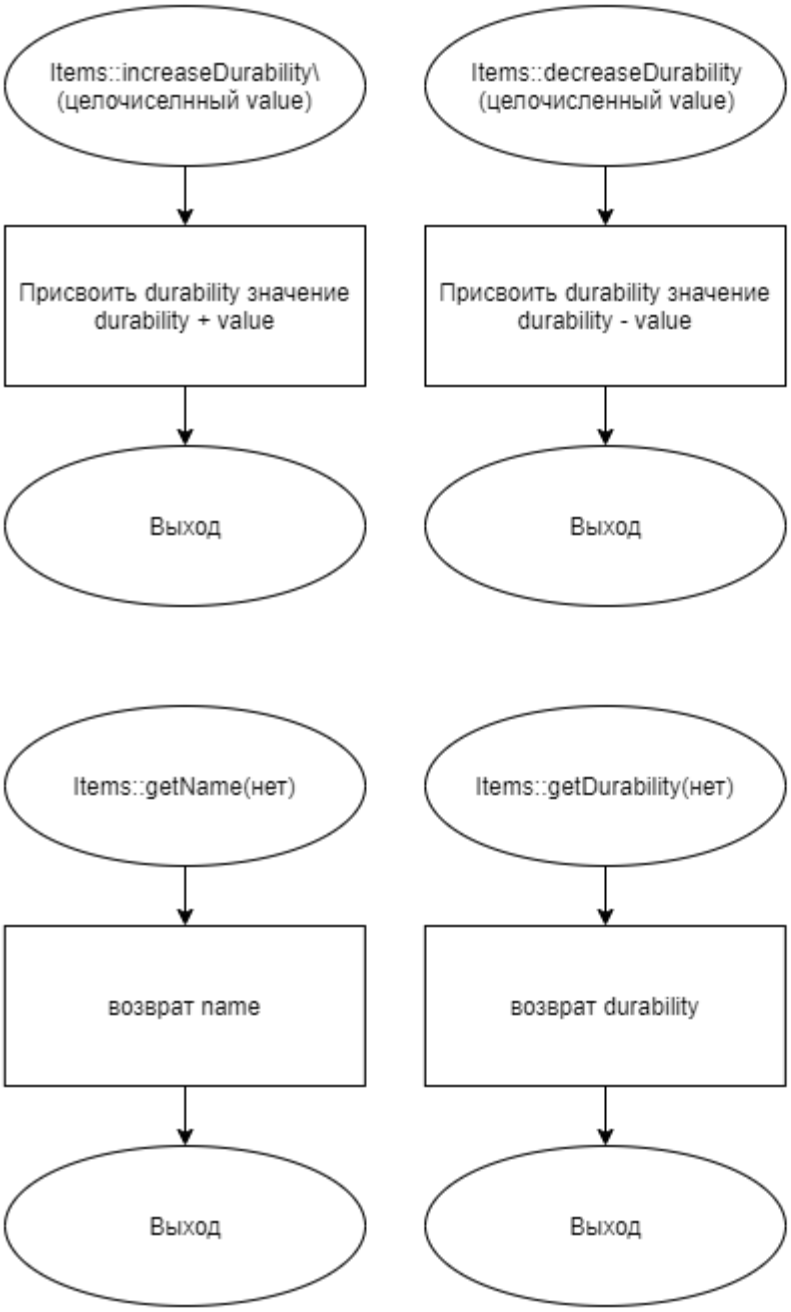

Рисунок 3 – Блок-схема алгоритма

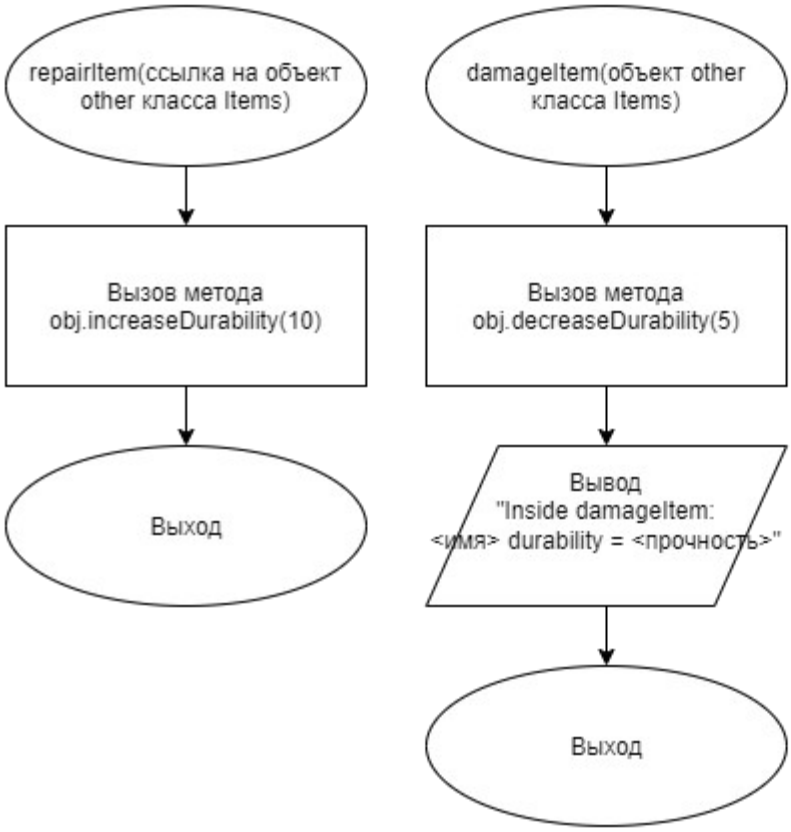

Рисунок 4 – Блок-схема алгоритма

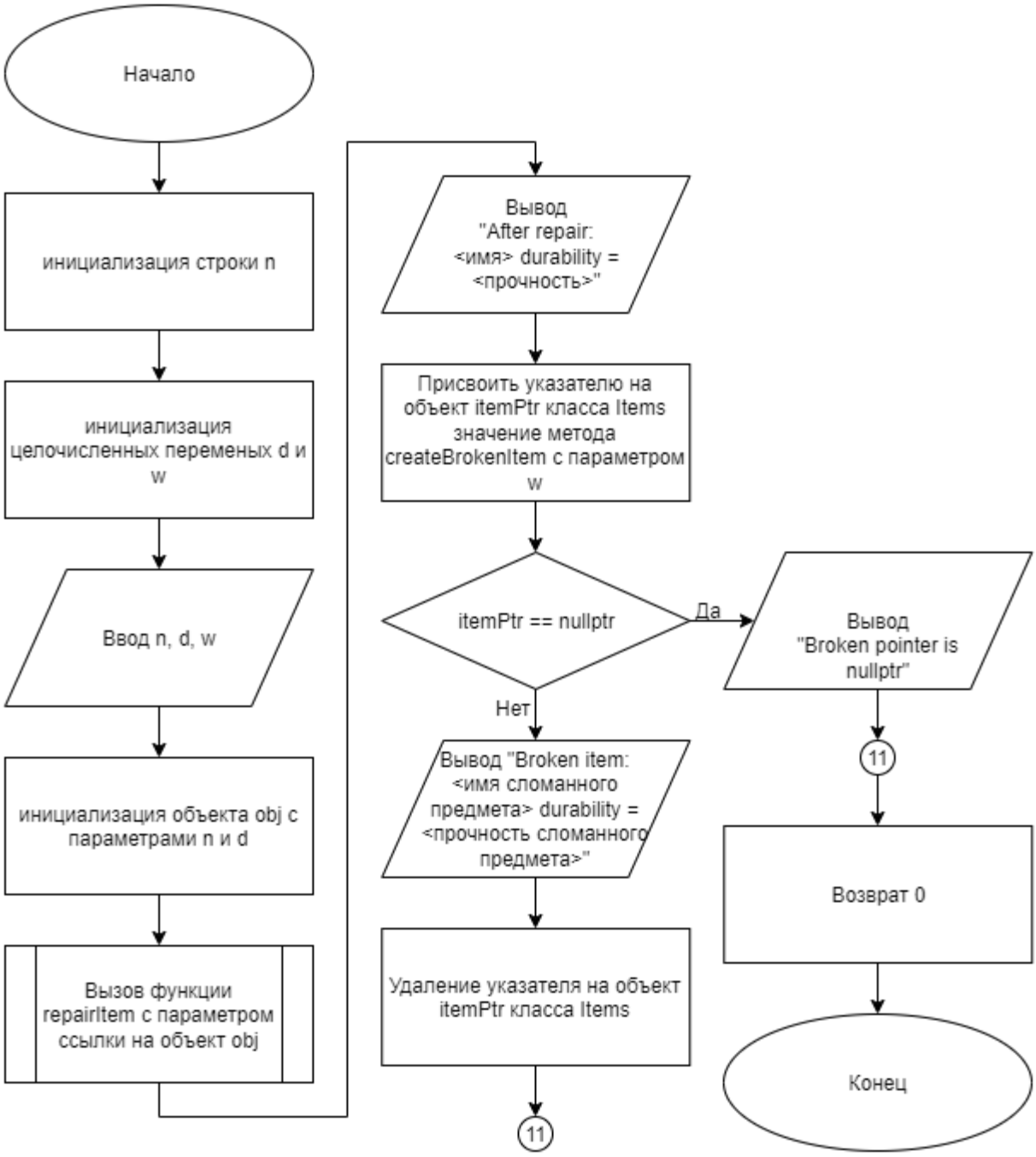

Рисунок 5 – Блок-схема алгоритма

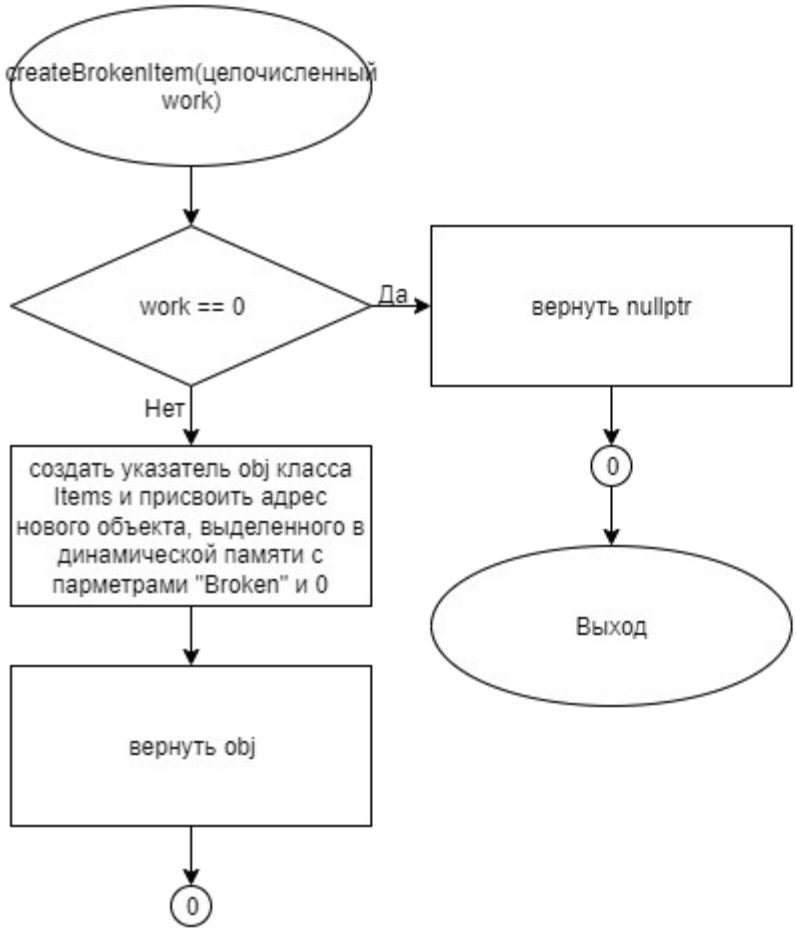

Рисунок 6 – Блок-схема алгоритма

5 КОД ПРОГРАММЫ

Программная реализация алгоритмов для решения задачи представлена ниже.

5.1 Файл Items.cpp

Листинг 1 – Items.cpp

```
#include "Items.h"

#include <iostream>

using namespace std;

Items::Items(){
   name = "Unknown";
   durability = 0;
   cout << "Item (Cnstr): " << name << endl;

}

Items::Items(string n, int d){
   name = n;
   durability = d;
   cout << "Item (ParamCnstr): " << name << endl;

}

Items::Items(const Items& other){
   name = other.name;
   durability = other.durability;
   cout << "Item (CopyCnstr): " << name << endl;
}

Items::~Items(){ cout << "~Item (Dcnstr): " << name << endl; }

void Items::increaseDurability(int value){ durability += value;}

void Items::decreaseDurability(int value){ durability -= value;}

string Items::getName(){ return name; }

int Items::getDurability(){ return durability; }

void repairItem(Items& obj){ obj.increaseDurability(10);}

void damageItem(Items obj){
```

```
      obj.decreaseDurability(5);
      cout << "Inside damageItem: " << obj.getName()
         << " durability = " << obj.getDurability() << endl;

   }

   Items* createBrokenItem(int work){
      if( work == 0){
         return nullptr;
      }
      else{
         Items *obj = new Items("Broken", 0);
         return obj;
      }
   }
```

5.2 Файл Items.h

Листинг 2 – Items.h

```
   #ifndef __ITEMS__H
   #define __ITEMS__H

   #include <string>

   using namespace std;

   class Items{
      string name;
      int durability;

      public:
         Items();
         Items(string n, int d);
         Items(const Items& other);
         ~Items();
         void increaseDurability(int value);
         void decreaseDurability(int value);
         string getName();
         int getDurability();
   };

   void repairItem(Items& obj);
   void damageItem(Items obj);
   Items* createBrokenItem(int work);

   #endif
```

5.3 Файл main.cpp

Листинг 3 – main.cpp

```
#include <iostream>
#include <string>
#include "Items.h"

using namespace std;

int main()
{
   string n;
   int d, w;
   cin >> n >> d >> w;
   Items obj(n, d);

   repairItem(obj);
   cout << "After repair: " << obj.getName()
      << " durability = " << obj.getDurability()
      << endl;

   damageItem(obj);
   cout << "After damage: " << obj.getName()
      << " durability = " << obj.getDurability()
      << endl;

   Items* itemPtr = createBrokenItem(w);
   if (itemPtr == nullptr)
      cout << "Broken pointer is nullptr";

   else
      cout << "Broken item: " << itemPtr -> getName()
      << " durability = " << itemPtr -> getDurability()
      << endl;

   delete itemPtr;
   return(0);
}
```

6 ТЕСТИРОВАНИЕ

Результат тестирования программы представлен в таблице 13.

Таблица 13 – Результат тестирования программы

Входные данные	Ожидаемые выходные данные	Фактические выходные данные
Sword 50 1	Item (ParamCnstr): Sword After repair: Sword durability = 60 Item (CopyCnstr): Sword Inside damageItem: Sword durability = 55 ~Item (Dcnstr): Sword After damage: Sword durability = 60 Item (ParamCnstr): Broken Broken item: Broken durability = 0 ~Item (Dcnstr): Broken ~Item (Dcnstr): Sword	Item (ParamCnstr): Sword After repair: Sword durability = 60 Item (CopyCnstr): Sword Inside damageItem: Sword durability = 55 ~Item (Dcnstr): Sword After damage: Sword durability = 60 Item (ParamCnstr): Broken Broken item: Broken durability = 0 ~Item (Dcnstr): Broken ~Item (Dcnstr): Sword

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 19 Единая система программной документации.

2. Методическое пособие студента для выполнения практических заданий, контрольных и курсовых работ по дисциплине «Объектно-ориентированное программирование» [Электронный ресурс] – URL: https://mirea.aco-avrora.ru/student/files/methodichescoe_posobie_dlya_laboratornyh_rabot_3.pdf (дата обращения 05.05.2021).

3. Приложение к методическому пособию студента по выполнению заданий в рамках курса «Объектно-ориентированное программирование» [Электронный ресурс]. URL: https://mirea.aco-avrora.ru/student/files/Prilozheniye_k_methodichke.pdf (дата обращения 05.05.2021).

4. Шилдт Г. C++: базовый курс. 3-е изд. Пер. с англ.. — М.: Вильямс, 2019. — 624 с.

5. Видео лекции по курсу «Объектно-ориентированное программирование» [Электронный ресурс]. АСО «Аврора».

6. Антик М.И. Дискретная математика [Электронный ресурс]: Учебное пособие /Антик М.И., Казанцева Л.В. — М.: МИРЭА — Российский технологический университет, 2018 — 1 электрон. опт. диск (CD-ROM).